When Does a Compact Scoped Symbol Table Pay Off? Bounded-Memory Tables Evaluated on Two Instrumented C Compilers

Adhyan Jain, Shubhi Singh, Akshith Venkataramana, Arjuman, and Bhuvaneshwari M
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*—A compiler that must run within a fixed memory budget needs a symbol table whose size is bounded in advance and whose behaviour at the bound is defined. We ask when a compact, interning table is worth its cost against the conventional design under such a bound. Two tables with the same storage discipline—one caller-supplied buffer, no allocation, refusal without state change—are compared: ARENA, the classic chained table over an entry arena, and STRATA, which stores each visible name once in packed form, keeps the innermost declaration in the name's record and saves hidden declarations on an undo stack. Workloads are the symbol-table calls logged by two instrumented C compilers, TinyCC and chibicc, on eight open-source projects, plus a parametric sweep over the shadowing factor and name length. Against our earlier table, METIS-X, both bounded tables cut memory by more than half, and ARENA alone accounts for most of that gain. Against ARENA, STRATA needs 16–18% less memory on the compiler traces, where there is almost no shadowing, and is slower on 7 of 8 of them (median 29%). A closed-form memory model predicts both tables' needs to within 1.8% and 0.1%; the saving grows to 65% with shadowing. Speed does not follow shadowing but name length: STRATA is the faster table for very short names and the slower one otherwise. The saving decides whether a translation unit fits a budget only in a narrow band: at most 23 of 169 units for any tested buffer size. Integrated into chibicc, both tables produce assembly identical to the original compiler on 158 of 158 units; neither improves whole-compiler time or memory, and STRATA costs up to four percent of compile time. We give a correctness argument with tests for the refusal guarantee and run both tables on an emulated Cortex-M3. We claim no new algorithm; the contribution is the measured answer to when the compact design pays off.**

***Index Terms*—symbol table, lexical scope, embedded compilers, memory budget, arena allocation, trace replay, empirical evaluation**

I. Introduction

A scoped symbol table maps identifiers to declarations and forgets a block's declarations when the block ends [1]. For a compiler that runs on a small device, or inside a tool with a fixed memory budget, three properties matter that general-purpose containers do not give: a memory bound fixed before compilation starts, no allocator calls during compilation, and a defined result when the bound is reached.

Two designs meet these requirements. The conventional one keeps one entry per declaration in an arena and links entries in hash chains; it is what small compilers have long done [2]. A compact one interns each name once and records shadowing on an undo stack [3], [4]; it stores less but does more work per operation. Which should a memory-constrained compiler use? The answer is not obvious, because the compact design saves memory only to the extent that names repeat among live declarations, and real C code may not repeat them.

This paper measures the answer. It continues the METIS-X project [5], a Robin Hood table [6] with one 32-byte slot per declaration, whose authors closed it after finding that its components were known, its traces came from regular expressions, and its design had no place in a compiler that interns identifiers first. An earlier version of the present work reported that a redesigned table used far less memory than METIS-X; a review asked whether that gain belonged to the design or merely to the absence of allocation. We therefore make the following contributions, none of which is a new algorithm.

1) A decomposition of the improvement over METIS-X into the part due to allocation-free arena storage and the part due to interning, undo entries and packed names (Section VI-A).
2) A closed-form memory model for both bounded tables, validated on a sweep, and the finding that the compact table's speed depends on name length, not on shadowing (Section VI-B).
3) A per-translation-unit analysis of when the saving decides whether a workload fits a buffer, with a demonstration on an emulated microcontroller (Section VI-C).
4) Workloads from two instrumented compilers, and integration of both tables into one of them with end-to-end measurements (Sections V and VI-D).
5) A correctness argument and tests for refusal without state change (Section IV).

II. Background: the Previous Table

We audited METIS-X at commit `7b3dacf`. All declarations of one name share a hash value and one probe cluster; the probe distance is an 8-bit field capped at 250, and an insertion beyond it is refused. On a trace that declares one name in 400 nested scopes METIS-X refused 149 declarations and returned 298 wrong lookups; no other table failed. Its lookup compares every live declaration of a name, and names over 12 bytes are separate heap blocks. Its earlier evaluation

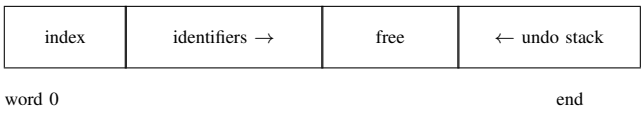


Fig. 1. STRATA's buffer. Identifier references are relative to the start of their region and undo references to the end of the buffer, so either boundary can move without patching stored references.

used traces extracted by regular expressions, included harness data in the heap figures, and timed each call with a clock as costly as the call.

III. Two Bounded-Memory Tables

Both tables keep all state in one array of 32-bit words that the caller may supply (fixed mode) or that grows by half through `realloc` (growable mode). Neither throws; in fixed mode neither allocates; a refused operation leaves the state unchanged.

A. ARENA: the conventional design

Bucket heads at the bottom of the buffer, a stack of entries above them, scope marks at the top. An entry (a 16-byte header and the name, unpacked) is created for every declaration and linked at the head of its hash chain, so the first match in a chain is the innermost declaration. Scope exit unlinks and pops the scope's entries.

B. STRATA: the compact design

The index is an open-addressing array with linear probing; a slot holds an 8-bit fingerprint [7] and a 24-bit reference. The identifier stack holds one record per distinct visible name: a value word, a meta word (name length in words, type tag, scope depth of the innermost declaration) and the name, packed five characters to a word in the manner of RADIX-50 [8]. The undo stack grows down from the end of the buffer (Fig. 1).

A lookup packs and hashes the query (MurmurHash3's 32-bit mixer [9]), probes, and reads the innermost declaration from the record; its cost does not depend on shadowing depth. A declaration of a new name appends a record, fills a slot and pushes that slot's index (one word) on the undo stack. A declaration that shadows pushes the record's current value and tag (three words) and overwrites them. Scope exit pops entries down to the scope's mark: a shadow entry restores what it saved; a first-declaration entry empties the recorded slot. Limits: names of 1–255 bytes, depth 16,383, buffer 64 MiB; an index rebuild is linear in the number of live names; the hash is unkeyed.

C. Memory model

Let N be the number of live declarations, D the number of distinct live names, L the name length in characters, d the number of open scopes, and $\sigma = N/D$ the *shadowing factor*. In bytes,

$$M_{STRATA} = D\,(8 + 4\lceil L/5\rceil) + 4\,I(D) + 4D + 12\,(N - D) + 8d, \quad (1)$$

$$M_{ARENA} = N\,(16 + 4\lceil L/4\rceil) + 4\,B(N) + 4d, \quad (2)$$

where $I(D)$ is the smallest $16 \cdot 2^k$ with $D \le 0.75\,I(D)$ and $B(N)$ the smallest $16 \cdot 2^k \ge N$. At $\sigma = 1$ the tables differ only through the header (8 against 16 bytes), the packing of the name and the index granularity; every additional declaration of an already live name costs 12 bytes in STRATA and a full entry in ARENA.

IV. Correctness of the Guarantees

Scope exit needs no tombstones. In linear probing without displacement a key is stored in the first empty slot of its probe path and never moves, so the path of a key consists of slots filled by older keys. The name removed at scope exit is the newest key in the index; it lies on no remaining key's path, and emptying its slot cuts no path. Rebuilding the index oldest first preserves the premise. This differs from general deletion, which needs tombstones or shifting [10], [11].

All-or-nothing. Each operation validates, reserves, then writes. The reservation either guarantees room for every word and slot the operation will write, or fails before anything is modified; in growable mode the only fallible step, the allocator call, comes first.

Tests. A digest of the complete state is taken before every operation and must be unchanged after every refusal, in 30 configurations: five buffer sizes, three workloads (random, deep shadowing, deep nesting), with the normal hash and with every key forced into one probe cluster. Each run is compared with a reference model and unwound to empty. Further tests use an allocator that fails after k calls, 5,000-deep shadowing of one name, and scope exit across index rebuilds; 1.86 million checks pass under AddressSanitizer and UndefinedBehaviorSanitizer. The argument is informal; nothing was machine-checked.

V. Methodology

Compiler traces. We instrumented two compilers. In TinyCC [2] (commit `43c7708`) we log `sym_find`, `sym_link`, `global_identifier_push` and every change of scope level. TinyCC interns identifiers and resolves through per-token pointers, so these are the calls its tables receive, not string lookups it performs. In chibicc [12] (commit `90d1f7f`) we log `find_var`, `push_scope`, `enter_scope` and `leave_scope`; chibicc resolves identifiers by string through one hash map per open scope, so a string-keyed scoped table is on its real resolution path. The compilers were run on cJSON, littlefs, zlib, Lua, FreeRTOS-Kernel, Mbed TLS, TinyCC itself and SQLite (Table I; chibicc compiles 158 of 170 units of seven of them). Each translation unit starts with a fresh table.

Sweep. Synthetic traces hold 4,096 live declarations at their peak, built from $4096/\sigma$ distinct names of exactly L characters

TABLE I
TINYCC TRACES

Trace	Units	Events	Declare	Lookup	Names	Peak live	Depth
cjson	2	20,874	5,247	9,194	1,366	1,067	10
freertos	9	37,875	10,346	17,417	2,226	1,505	16
littlefs	2	21,080	3,835	12,230	1,001	732	16
lua	34	188,509	58,619	86,887	3,622	1,365	14
mbedtls	105	983,008	319,886	438,876	7,622	2,302	14
sqlite	1	268,960	27,072	172,426	7,801	4,245	22
tinycc	1	89,135	13,776	54,623	6,704	5,509	27
zlib	15	58,058	16,341	27,405	1,041	571	23

declared in nested scopes, with five lookups per declaration; $\sigma \in \{1, 1.25, 1.5, 2, 3, 4, 8\}$ and $L \in \{6, 12, 24\}$.

Tables. STRATA; ARENA; METIS-X and its two baselines, unmodified; a hash map per open scope (chibicc's own design); the undo-log algorithm of LLVM's `ScopedHashTable` [4] on `std::unordered_map` and on a Swiss-table node map [13]; and an interning Swiss flat map, mirroring the split between interning and resolution in Clang [14].

Measurements. Every result is compared with a reference model built on `std::map`. For the two bounded tables memory is the smallest fixed buffer, found by bisection to 64 bytes, with which a trace or unit replays without a refusal; for the others it is peak heap through an allocator hook. Time: 31 rounds after two warm-up rounds; in each round every table replays the whole trace once, in an order rotated from round to round; for two tables we report the median per-round relative difference with a 95% bootstrap interval and call it *established* when the interval excludes zero. Runs were pinned to one core of an Intel Core Ultra 7 258V under Linux (WSL 2), GCC 11.4, `-O2`.

VI. RESULTS

All 96 of 96 table–trace pairs on the TinyCC traces agree with the reference model, as do all pairs on the chibicc traces.

A. Decomposition of the gain over METIS-X

Table II places ARENA between METIS-X and STRATA. ARENA's smallest buffer is already 55–77% below METIS-X's peak heap, and it replays the traces 11–36% faster. STRATA's buffer is a further 16–18% smaller (15.8–156.5 KiB, or 27–36 bytes per live declaration), but STRATA is established slower than ARENA on 7 of 8 traces, by 3–63% (median 29%). Against METIS-X itself, STRATA uses 23–76% less heap with 98.7–99.9% fewer allocator calls and is established faster on 7 of 8 traces.

At the memory peak the number of live declarations equals the number of distinct live names on all TinyCC traces but one: σ is 1. Interning therefore saves nothing there; STRATA's advantage is the smaller record and packed names, and its time cost is packing the query and probing an open-addressing index instead of following a short chain that stores full hashes. Most of the improvement over METIS-X is thus obtained by allocation-free storage, not by the compact structure.

TABLE II
METIS-X PEAK HEAP AND THE SMALLEST FIXED BUFFER OF THE TWO BOUNDED TABLES (KIB); REDUCTION (%) OF STRATA AGAINST ARENA IN MEMORY AND IN REPLAY TIME, WITH 95% INTERVAL. NEGATIVE: STRATA IS WORSE.

Trace	METIS-X	arena	STRATA	mem.	time	95% CI
cjson	103	35.9	30.0	16	-63	[-66, -54]
freertos	217	53.6	44.0	18	-28	[-37, -24]
littlefs	107	24.2	20.0	18	-35	[-42, -25]
lua	107	43.4	35.9	17	-33	[-39, -23]
mbedtls	234	96.9	80.0	17	-31	[-33, -24]
sqlite	451	169.4	140.4	17	-3	[-7, 6]
tinycc	423	189.8	156.5	18	-23	[-27, -20]
zlib	53	18.9	15.8	17	-14	[-31, -10]

TABLE III
SWEEP: REDUCTION (%) OF STRATA AGAINST ARENA IN MINIMUM BUFFER AND IN REPLAY TIME (PAIRED MEDIANS), BY SHADOWING FACTOR σ AND NAME LENGTH L. NEGATIVE: STRATA IS WORSE.

σ	mem. L=6	mem. L=12	mem. L=24	time L=6	time L=12	time L=24
1.00	0	0	9	16	-26	-40
1.20	6	8	18	25	-7	7
1.50	24	25	33	24	-36	-29
2.00	29	31	41	33	-15	-27
3.00	40	44	53	22	-27	-33
4.00	43	47	57	32	11	-34
8.00	50	55	65	32	-10	-24

B. When the compact design pays off

Table III gives the sweep. The model of Section III-C predicts the measured minimum buffers to within 1.8% for STRATA and 0.1% for ARENA. *Memory.* At $\sigma = 1$ the saving is 0–9%: it depends on name length and on where the number of names falls between two index sizes (the compiler traces, with mixed lengths, show 16–18%). At $\sigma = 2$ it is 29–41%, and it grows monotonically to 65% at $\sigma = 8$. *Time.* We expected a shadowing factor beyond which STRATA would also be faster; the data do not show one. Time follows name length instead: with 6-character names STRATA is established faster at 7 of 7 shadowing factors, and with 12- and 24-character names it is established slower at 11 of 14 points and faster at 2. The values are not monotone in σ, which we attribute to cache and index-size effects the sweep does not control; only the pattern by name length is consistent. The rule that follows: choose the compact table for memory, in proportion to shadowing; do not expect it to be faster unless identifiers are very short.

C. Does the saving let a workload fit?

For each of the 169 translation units of the TinyCC traces we determined the smallest buffer of each table (Table IV). Per unit, STRATA needs 2–18% less than ARENA (median 17%). The saving changes the outcome only when the budget lies between the two requirements: at 16 KiB, 51 units fit STRATA and 28 fit ARENA, the largest gap we observed; at 64 KiB and above the counts differ by at most two. On an emulated Cortex-M3 with 64 KiB of RAM and a 48 KiB table buffer, STRATA replays the FreeRTOS-Kernel trace completely while ARENA refuses 420 operations; with 8 KiB

TABLE IV
NUMBER OF THE 169 TINYCC TRANSLATION UNITS WHOSE SYMBOL WORKLOAD FITS A BUFFER OF THE GIVEN SIZE

Buffer (KiB)	4	8	16	24	32	48	64	96	128	192
STRATA fits	0	2	51	90	107	132	147	167	167	169
arena fits	0	1	28	83	100	113	146	165	167	169
only STRATA	0	1	23	7	7	19	1	2	0	0

TABLE V
REPLAY TIME ON THE TINYCC TRACES, NS PER TRACE OPERATION, MEDIAN OF 31 ROUNDS (LOWEST IN BOLD)

Trace	STRATA	arena+chains	METIS-X	map/scope	std+undo	Swiss+undo	Swiss int.
cjson	53.4	**33.1**	49.7	60.8	55.0	53.2	39.6
freertos	42.9	**33.1**	48.6	52.6	43.0	46.6	35.1
littlefs	44.2	**32.0**	49.5	69.3	49.5	44.5	36.6
lua	43.5	33.0	47.6	51.2	46.0	46.4	**32.3**
mbedtls	45.5	**35.4**	55.6	51.1	49.6	52.8	38.1
sqlite	29.0	28.9	32.7	60.3	29.4	30.6	**25.0**
tinycc	33.0	26.5	35.0	60.1	33.3	31.5	**26.4**
zlib	33.7	29.3	37.5	49.8	39.8	40.1	**28.7**

more both succeed. The reduction is therefore decisive for a design whose table budget is fixed close to the workload's need, and immaterial otherwise.

D. Inside a compiler

We routed chibicc's variable scopes through each bounded table. Both builds produce assembly identical to the original compiler on all 158 units the original compiles. Whole-project compile time over 21 interleaved rounds changes by -3.7% to 0.8% with STRATA (established slower on 3 of 7 projects, faster on none) and by -2.2% to 2.8% with ARENA (no established difference); peak resident memory changes by a median of 0.14% per unit. chibicc spends little of its time in scope lookups and never frees its tokens and syntax trees, so neither table gives an end-to-end benefit in this hosted compiler. Replaying chibicc's own scope calls, where the tables are measured alone, STRATA is established faster than chibicc's per-scope maps on 4 of 7 traces and established slower than ARENA on 4.

E. Other tables and stress cases

Table V gives replay times on the TinyCC traces. The interning flat map is faster than STRATA on 8 of 8 traces (median 21%) and uses the most memory of all tables. The interquartile range of raw replay times has a median of 16% of the median, which is why only paired differences are reported. With every key forced into one probe cluster a lookup among 4,096 names takes 1588 ns instead of 14 ns: results stay correct but there is no defence against adversarial keys. On the emulated Cortex-M3, STRATA replayed 5 traces from buffers of 16–44 KiB with host-identical results, 792 bytes of stack and 2,557 bytes of code.

VII. RELATED WORK

A single hash table with an auxiliary stack unwound at scope exit is described in compiler textbooks [1], [3] and implemented in LLVM's `ScopedHashTable` [4]; chained tables over arenas popped at scope exit are equally standard, and arena allocation by lifetime is long established in compilers [15]. Clang separates an identifier table from a resolver [14]; TinyCC keeps a per-token symbol pointer with symbol stacks [2]; chibicc uses a hash map per scope [12]. Character packing goes back to RADIX-50 [8]. We found no published comparison of these designs under a hard memory bound on compiler-generated events, but our search of prior work was not systematic.

VIII. THREATS TO VALIDITY

Workloads. Two compilers, C only. Languages with heavier shadowing or longer names (C++ in particular) may sit elsewhere in the (σ, L) plane; the sweep covers that plane only synthetically. Three Mbed TLS files that TinyCC rejects and twelve units that chibicc rejects are missing. *Hardware.* All timings come from one laptop under a hypervisor; the microcontroller results are from an instruction-level emulator and include no timing. *Baselines.* All tables except METIS-X's were written by us after known designs; they are not the production code of LLVM or Clang. *Integration.* In chibicc only variable scopes were replaced, and file-scope declarations are kept in a second table instance to answer chibicc's global lookups, which slightly penalises both bounded tables. *Causes.* We did not separate the cost of packing from that of open addressing, and the sweep's time results are not monotone in the shadowing factor for reasons we did not isolate. *Correctness.* The argument of Section IV is informal.

IX. CONCLUSION

Under a hard memory bound, the first-order gain over a per-declaration heap-allocating table comes from allocation-free arena storage, which the conventional chained design provides and provides faster. A compact interning design saves roughly a sixth more memory on ordinary C code, where live names are almost never shadowed, at a clear cost in time. Its saving grows with the shadowing factor as a simple model predicts; its speed does not, and depends on identifier length instead. The saving determines whether a workload fits only when the budget lies within that margin of the need. Inside a hosted compiler neither design improves time or memory end to end. For a memory-constrained C compiler the conventional design is therefore the default, and the compact one is justified only when the table budget is the binding constraint or the language shadows heavily.

ACKNOWLEDGMENT

An AI coding assistant (Claude, Anthropic) was used extensively for the implementation, the experiments and the drafting of this paper. The authors are responsible for verifying the code and the claims.

Data and Code Availability

Source code, traces, raw measurements (run `20261011-165103-v3`) and the scripts that generate every table and figure are in the project repository. METIS-X, chibicc, TinyCC and the Swiss-table library are fetched at pinned versions and are not redistributed.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[2] F. Bellard and TinyCC contributors, "Tiny C Compiler," https://repo.or.cz/tinycc.git, commit 43c7708.

[3] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge University Press, 1998.

[4] LLVM Project, "ScopedHashTable.h, LLVM ADT library," https://llvm.org/doxygen/ScopedHashTable_8h_source.html.

[5] A. Jain and S. Singh, "METIS: symbol-table research repository, commit 7b3dacf," https://github.com/adhyan-jain/Metis, 2026.

[6] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annual Symposium on Foundations of Computer Science (FOCS)*, 1985.

[7] Abseil, "Swiss tables design notes," https://abseil.io/about/design/swisstables.

[8] Wikipedia contributors, "DEC RADIX 50," https://en.wikipedia.org/wiki/DEC_RADIX_50.

[9] A. Appleby, "MurmurHash3, SMHasher," https://github.com/aappleby/smhasher.

[10] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[11] M. A. Bender, B. C. Kuszmaul, and W. Kuszmaul, "Linear probing revisited: Tombstones mark the demise of primary clustering," in *Proc. 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, 2021, pp. 1171–1182.

[12] R. Ueyama, "chibicc: A small C compiler," https://github.com/rui314/chibicc, commit 90d1f7f, 2020.

[13] G. Popovitch, "The parallel hashmap, version 1.4.1," https://github.com/greg7mdp/parallel-hashmap.

[14] LLVM Project, "IdentifierTable.cpp and IdentifierResolver.h, Clang," https://github.com/llvm/llvm-project/tree/main/clang.

[15] D. R. Hanson, "Fast allocation and deallocation of memory based on object lifetimes," *Software: Practice and Experience*, vol. 20, no. 1, pp. 5–12, 1990.